

Assignment 6: Public Key Cryptography DESIGN.pdf:

Description of Program:

This assignment consists of three implemented programs. The keygen program is in charge of key generation, producing RSA public and private key pairs. The encrypt program encrypts files using a public key, and the decrypt program decrypts the encrypted files using the corresponding private key. The program is written using two implemented libraries. One of the libraries holds functions relating to the mathematics behind RSA, and the other library itself contains implementations of routines for RSA.

Files to be included in directory “asgn6” + pseudocode/structure:

- numtheory.c: This contains the implementations of the number theory functions.
 - Mod_inverse
 - Computes the inverse i of a modulo n . In the event that a modular inverse cannot be found, set i to 0. Use aux variables.
 - $(r, r') \leftarrow (n, a)$
 - $2 \ (t, t') \leftarrow (0, 1)$
 - while $r' \neq 0$
 - $q \leftarrow r/r'$
 - $(r, r') \leftarrow (r', r - q \times r')$
 - $(t, t') \leftarrow (t', t - q \times t')$

if $r > 1$

return no inverse

■ if $t < 0$

• $t \leftarrow t+n$

■ return t

○ Gcd

■ Computes the greatest common divisor of a and b , storing the value of the computed divisor in d

■ while $b \neq 0$

• $2t \leftarrow b$

• $b \leftarrow a \bmod b$

• $a \leftarrow t$

■ return a

○ Make_prime

■ Generates a new prime number stored in p . The generated prime should be at least $bits$ number of bits long. The primality of the generated number should be tested using `is_prime()` using `iters` number of iterations.

■ while p isn't prime

• Generate p

■ return p

○ Is_prime

- Conducts the Miller-Rabin primality test to indicate whether or not n is prime using its number of Miller-Rabin iterations. This function is needed when creating the two large primes p and q in RSA, verifying if a large integer is a prime.

- write $n-1 = 2^s r$ such that r is odd

- for $i \leftarrow 1$ to k

- choose random $a \in \{2, 3, \dots, n-2\}$

- $y = \text{POW_ER_MOD}(a, r, n)$

- if $y \neq 1$ and $y \neq n-1$

- $j \leftarrow 1$

- while $j \leq s-1$ and $y \neq n-1$

- $y \leftarrow \text{POW_ER_MOD}(y, 2, n)$

- if $y == 1$

- return FALSE

- $j \leftarrow j+1$

- if $y \neq n-1$

- return FALSE

- return TRUE

- Pow_mod

- Performs fast modular exponentiation, computing base raised to the exponent power modulo modulus, and storing the computed result in out.
 - $v \leftarrow 1$
 - $p \leftarrow a$
 - while $d > 0$
 - if $\text{OD } D(d)$
 - $v \leftarrow (v \times p) \bmod n$
 - $p \leftarrow (p \times p) \bmod n$
 - $d \leftarrow bd/2c$
 - return v
- numtheory.h: This specifies the interface for the number theory functions.
- randstate.c: This contains the implementation of the random state interface for the RSA library and number theory functions.
 - Initializes the global random state named state with a Mersenne Twister algorithm, using seed as the random seed. Calls to randinit_mt() and gmp_randseed_ui().
 - Clears and frees all memory used by the initialized global random state named state. Single call to gmp_randclear().
- randstate.h: This specifies the interface for initializing and clearing the random state.
- rsa.c: This contains the implementation of the RSA library.
 - Rsa_verify

- Performs RSA verification, returning true if signature s is verified and false otherwise. Verification is the inverse of signing. Let $t = V(s) = se \pmod{n}$. The signature is verified if and only if t is the same as the expected message m .
 - `powermod(s,e,n)`
- `Rsa_sign`
 - Performs RSA signing, producing signature s by signing message m using private key d and public modulus n .
Signing with RSA is defined as $S(m) = s = m^d \pmod{n}$.
 - `powermod(m, d, n)`
- `Rsa_decrypt_file`
 - Decrypts the contents of `infile`, writing the decrypted contents to `outfile`. The data in `infile` should be decrypted in blocks to mirror how `rsa_encrypt_file()` encrypts in blocks.
 - Calculate the block size k . This should be $k = \frac{b(\log_2(n)-1)}{8c}$.
 - Dynamically allocate an array that can hold k bytes. This array should be of type `(uint8_t *)` and will serve as the block.
 - While there are still unprocessed bytes in `infile`:
 - Scan in a hexstring, saving the hexstring as a `mpz_t`
 - c. Remember, each block is written as a hexstring with a trailing newline when encrypting a file.

- Using `mpz_export()`, convert `c` back into bytes, storing them in the allocated block. Let `j` be the number of bytes actually converted. You will want to set the order parameter of `mpz_export()` to 1 for most significant word first, 1 for the endian parameter, and 0 for the nails parameter.
 - Write out `j-1` bytes starting from index 1 of the block to outfile. This is because index 0 must be prepended 0xFF. Do not output the 0xFF.
- `Rsa_decrypt`
 - Performs RSA decryption, computing message `m` by decrypting ciphertext `c` using private key `d` and public modulus `n`. Decryption with RSA is defined as $D(c) = m = c^d \pmod{n}$.
- `Rsa_encrypt_file`
 - Encrypts the contents of `infile`, writing the encrypted contents to outfile. The data in `infile` should be encrypted in blocks. We are working modulo `n`, which means that the value of the block of data we are encrypting must be strictly less than `n`.
 - Calculate the block size `k`. This should be $k = \lfloor \frac{b(\log_2(n)-1)}{8c} \rfloor$.

- Dynamically allocate an array that can hold k bytes. This array should be of type (uint8_t *) and will serve as the block.
- Set the zeroth byte of the block to 0xFF. This effectively prepends the workaround byte that we need.
- While there are still unprocessed bytes in infile:
 - (a) Read at most k-1 bytes in from infile, and let j be the number of bytes actually read. Place the read bytes into the allocated block starting from index 1 so as to not overwrite the 0xFF.
 - (b) Using mpz_import(), convert the read bytes, including the prepended 0xFF into an mpz_t m. You will want to set the order parameter of mpz_import() to 1 for most significant wordfirst, 1 for the endian parameter, and 0 for the nails parameter.
 - Encrypt m with rsa_encrypt(), then write the encrypted number to outfile as a hexstring followed by a trailing newline.
- Rsa_encrypt
 - Performs RSA encryption, computing ciphertext c by encrypting message m using public exponent e and modulus n. Encryption with RSA is defined as $E(m) = c = me \pmod n$.
- Rsa_read_priv

- Reads a private RSA key from pvfile. The format of a private key should be n then d, both of which should have been written with a trailing newline. Both these values should have been written as hexstrings.
- Rsa_make_priv
 - Writes a private RSA key to pvfile. The format of a private key should be n then d, both of which are written with a trailing newline. Both these values should be written as hexstrings.
- Rsa_read_pub
 - Reads a public RSA key from pbfile. The format of a public key should be n, e, s, then the username, each of which should have been written with a trailing newline. The values n, e, and s should have been written as hexstrings.
- Rsa_write_pub
 - Writes a public RSA key to pbfile. The format of a public key should be n, e, s, then the username, each of which are written with a trailing newline. The values n, e, and s should be written as hexstrings.
- Rsa_make_pub
 - Creates parts of a new RSA public key: two large primes p and q, their product n, and the public exponent e.

- Begin by creating primes p and q using `make_prime()`. We first need to decide the number of bits that go to each prime respectively such that $\log_2(n) \geq \text{nbits}$. Let the number of bits for p be a random number in the range $[\text{nbits}/4, (3 \times \text{nbits})/4)$. The remaining bits will go to q . The number of Miller-Rabin iterations is specified by `iters`.
- Next, compute $\phi(n) = (p-1)(q-1)$.
- We now need to find a suitable public exponent e . In a loop, generate random numbers of around `nbits` using `mpz_urandomb()`. Compute the `gcd()` of each random number and the computed totient. Stop the loop you have found a number coprime with the totient: that will be the public exponent.
- `rsa.h`: This specifies the interface for the RSA library.
- `decrypt.c`: This contains the implementation and `main()` function for the decrypt program.
 - Parse command-line options using `getopt()` and handle them accordingly.
 - Open the private key file using `fopen()`. Print a helpful error and exit the program in the event of failure.
 - Read the private key from the opened private key file.
 - If verbose output is enabled print the following, each with a trailing newline, in order:

- the public modulus n
 - the private key e
- Decrypt the file using `rsa_decrypt_file()`.
- Close the private key file and clear any `mpz_t` variables you have used.
- `encrypt.c`: This contains the implementation and `main()` function for the `encrypt` program.
 - Parse command-line options using `getopt()` and handle them accordingly.
 - Open the public key file using `fopen()`. Print a helpful error and exit the program in the event of failure.
 - Read the public key from the opened public key file.
 - Convert the username that was read in to an `mpz_t`. This will be the expected value of the verified signature. Verify the signature using `rsa_verify()`, reporting an error and exiting the program if the signature couldn't be verified.
 - Encrypt the file using `rsa_encrypt_file()`.
 - Close the public key file and clear any `mpz_t` variables you have used.
- `keygen.c`: This contains the implementation and `main()` function for the `keygen` program.
 - Parse command-line options using `getopt()` and handle them accordingly.

- Open the public and private key files using `fopen()`. Print a helpful error and exit the program in the event of failure.
- Using `fchmod()` and `fileno()`, make sure that the private key file permissions are set to 0600, indicating read and write permissions for the user, and no permissions for anyone else.
- Initialize the random state using `randstate_init()`, using the set seed.
- Make the public and private keys using `rsa_make_pub()` and `rsa_make_priv()`, respectively.
- Get the current user's name as a string. You will want to use `getenv()`.
- Convert the username into an `mpz_t` with `mpz_set_str()`, specifying the base as 62. Then, use `rsa_sign()` to compute the signature of the username.
- Write the computed public and private key to their respective files.
- Close the public and private key files, clear the random state with `randstate_clear()`, and clear any `mpz_t` variables you may have used.
- Makefile
 - CC = clang must be specified.
 - • CFLAGS = -Wall -Wextra -Werror -Wpedantic must be specified.
 - • pkg-config to locate compilation and include flags for the GMP library must be used.

- • make must build the encrypt, decrypt, and keygen executables, as should make all.
- • make decrypt should build only the decrypt program.
- • make encrypt should build only the encrypt program.
- • make keygen should build only the keygen program.
- • make clean must remove all files that are compiler generated.
- • make format should format all your source code, including the header files.
- README.md
 - In proper Markdown syntax.
 - Describe how to use your program and Makefile
 - Lists and explains any command-line options that program accepts.
 - Reports any false positives by scan-build
 - Notes down any known bugs or errors in this file as well for the graders.
- DESIGN.pdf
 - This document must be a proper PDF.
 - Describes design and design process for program with enough detail such that a sufficiently knowledgeable programmer would be able to replicate implementation.